Exercício Prático 2 — Aprendizado Supervisionado

Predição de inadimplência de crédito · Aula IAML 05 (Prática 2) · Inteligência Artificial e Machine Learning

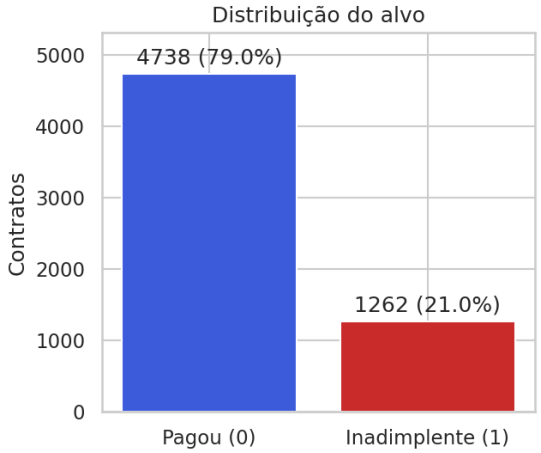
Aluno(a): _____ RA: _____

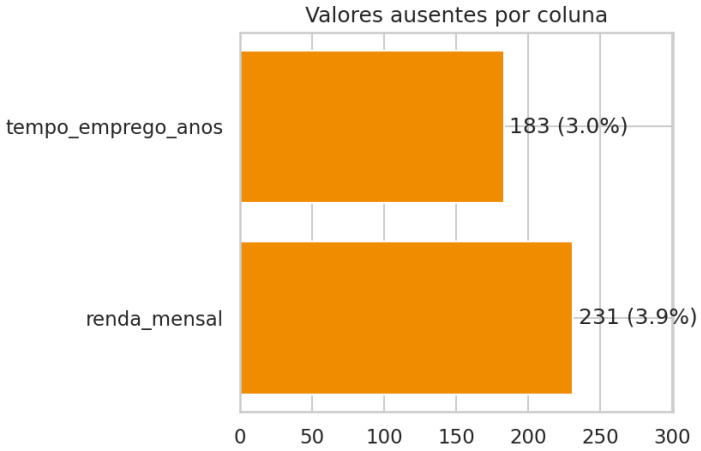
Dados sintéticos (`data/credito.csv`, gerado por `gerar_dados.py`, semente 42). Todos os números foram produzidos por `analise_credito.py` e são reprodutíveis. Ambiente: Python 3.12, scikit-learn 1.8.0 (a aula fixa 1.9.1; os resultados de churn e imóveis coincidem com os da aula).

Parte 1 — Análise exploratória

Proporção de inadimplentes. Dos 6000 contratos, 21,0% são inadimplentes (1) e 79,0% pagaram (0). As classes são desbalanceadas (cerca de 1 para 4), então a acurácia sozinha engana: aprovar todos os pedidos já acertaria cerca de 79%.

Valores ausentes. Apenas duas colunas: `renda_mensal` com 231 ausentes (3,9%) e `tempo_emprego_anos` com 183 (3,0%). São poucos, então imputamos (mediana) dentro do Pipeline em vez de descartar linhas.



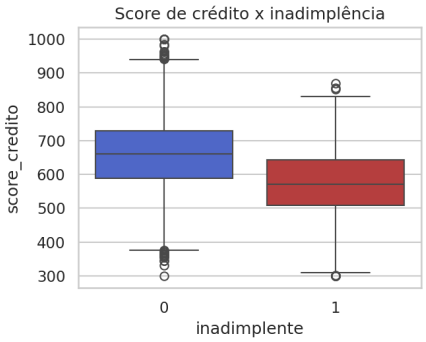


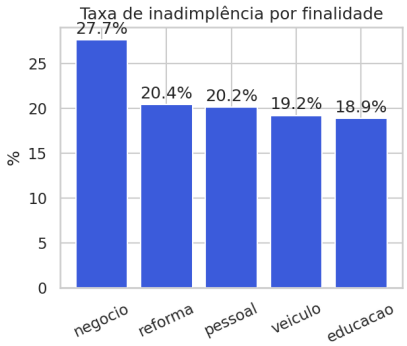
Como a inadimplência varia.

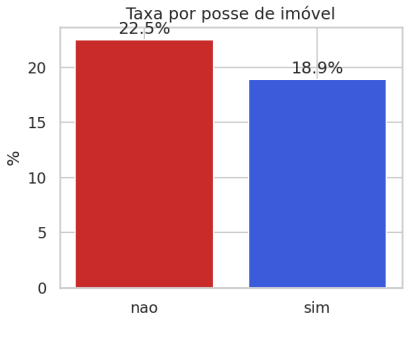
• **Score:** relação forte e monotônica. Faixa 300–500: 47,8%; 500–600: 30,8%; 600–700: 16,8%; 700–800: 10,2%; acima de 800: 1,8%. É o melhor preditor individual (correlação de –0,32 com o alvo).

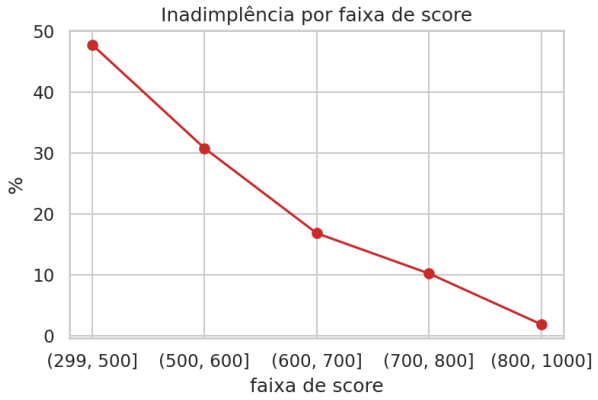
• **Finalidade:** *negócio* se destaca (27,7%); as demais ficam entre 18,9% e 20,4%.

• **Posse de imóvel:** quem não possui imóvel inadimpliu mais (22,5% contra 18,9%); efeito moderado.









Alguma coluna deve ser descartada? Sim: `id_contrato`. É um identificador único sem poder preditivo; mantê-lo permitiria ao modelo (sobretudo às árvores) decorar contratos individuais, gerando overfitting. O alvo `inadimplente` também fica fora de X. As demais colunas existem no momento da solicitação do empréstimo, então não há indício de vazamento de dados.

Parte 2 — Modelagem

Separação 80/20, estratificada, `random_state=42`: 4800 contratos de treino (21,0% inadimplentes) e 1200 de teste (21,0%). O Pipeline (ColumnTransformer) imputa pela mediana e padroniza as numéricas; imputa pela moda e aplica one-hot nas categóricas (`handle_unknown="ignore"`). Medianas, médias e categorias são aprendidas só com o treino de cada fold, o que evita vazamento. Validação cruzada de 5 folds, métrica ROC AUC:

Algoritmo	ROC AUC (média ± desvio)
Regressão Logística	**0,7781 ± 0,0224**
Árvore de Decisão	0,7126 ± 0,0216
Random Forest	0,7613 ± 0,0235
Gradient Boosting	0,7547 ± 0,0217

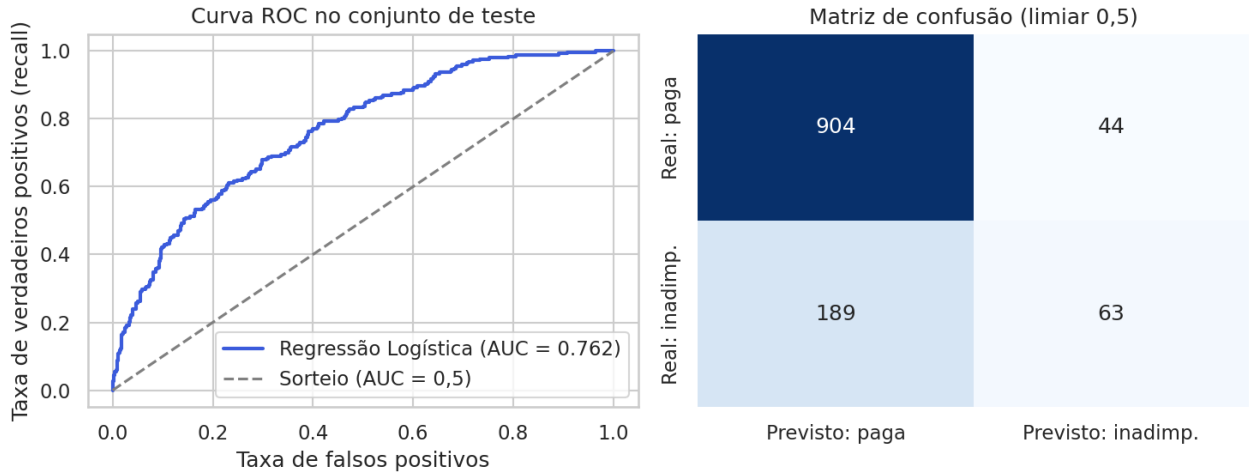
Escolhido: Regressão Logística. A vantagem sobre Random Forest e Gradient Boosting é pequena em relação ao desvio entre folds (~0,022). A relação entre as features e o risco é aproximadamente linear nestes dados, então os modelos mais complexos só acrescentaram variância (mesma lição do churn na aula). A regressão logística também é interpretável, o que importa em crédito (Parte 6). A Árvore de Decisão (profundidade 5) foi a pior.

Parte 3 — Avaliação no conjunto de teste

O teste foi usado uma única vez, com o modelo já escolhido e o limiar padrão 0,5:

Acurácia	Precisão	Recall	F1	ROC AUC
80,6%	58,9%	25,0%	0,351	0,762

	Previsto: paga	Previsto: inadimplente
Real: pagou	904 (VN)	44 (FP)
Real: inadimplente	189 (FN)	63 (VP)



Leitura. A acurácia de 80,6% parece boa, mas engana: 79,0% dos contratos pagam, então aprovar todos já daria 79,0%. O que pesa é o recall de apenas 25,0%: com o limiar 0,5 o modelo deixa passar 189 dos 252 inadimplentes. O ROC AUC de 0,762 mostra boa capacidade de ordenar o risco (e está longe de 0,95, então não há suspeita de vazamento); o problema é o corte de 0,5, conservador demais para este negócio.

Falso positivo (FP): o modelo prevê inadimplência e a fintech recusa, mas o cliente pagaria. É um **bom cliente recusado**; o custo é o lucro não obtido (R$ 1.500).

Falso negativo (FN): o modelo prevê que o cliente pagará e a fintech aprova, mas ele não paga. É um **inadimplente aprovado**; o custo é o prejuízo (R$ 8.000).

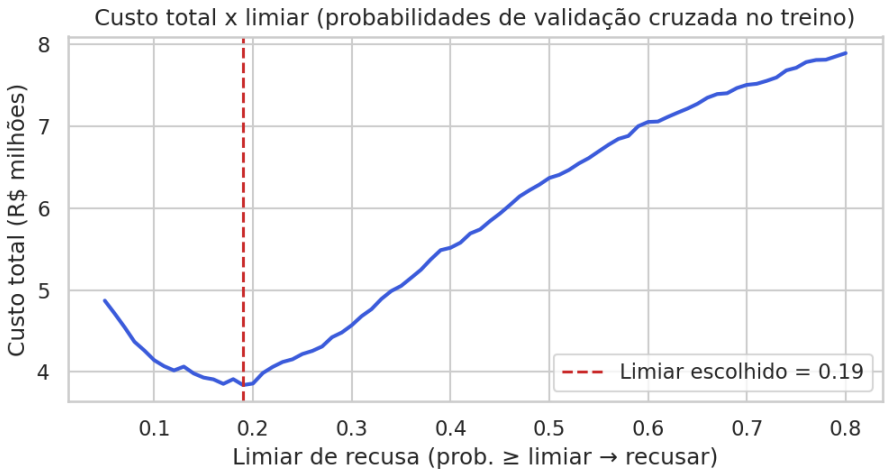
Qual é mais caro? O falso negativo: cada um custa 8.000 / 1.500 ≈ 5,3 vezes mais que um falso positivo. Por isso o recall pesa mais que a precisão aqui, e o limiar deve ficar abaixo de 0,5.

Parte 4 — Limiar de decisão

As probabilidades vieram de `cross_val_predict(pipeline, X_treino, y_treino, cv=5, method="predict_proba")`: cada previsão é feita por um modelo que não viu aquele contrato, e o teste não foi tocado. Regra: recusar se probabilidade ≥ limiar. Custo total = 8.000 × FN + 1.500 × FP.

Limiar	Precisão	Recall	Recusados	FN	FP	Custo (treino, CV)
0,10	28,7%	91,2%	3208	89	2287	R$ 4.142.500
0,15	33,3%	82,2%	2490	180	1660	R$ 3.930.000
0,19	**37,8%**	**75,9%**	**2030**	**243**	**1263**	**R$ 3.838.500**
0,30	47,2%	55,0%	1175	455	620	R$ 4.570.000
0,50	60,9%	24,1%	399	767	156	R$ 6.370.000

Referências no treino: aprovar todos custaria R$ 8.080.000 e recusar todos, R$ 5.685.000.



Recomendação: limiar de 0,19 (varredura de 0,05 a 0,80 em passos de 0,01, minimizando o custo nas probabilidades de validação cruzada). Há um argumento teórico que corrobora: se as probabilidades forem bem calibradas, recusar compensa quando p × 8.000 > (1 – p) × 1.500, ou seja, p > 1.500 / 9.500 ≈ 0,158. O valor empírico (0,19) fica próximo, e a curva de custo é achatada entre ~0,15 e ~0,22, então a escolha não é frágil.

Custo no conjunto de teste com este limiar: R$ 1.075.000 (71 FN e 338 FP; precisão 34,9%, recall 71,8%). Para comparar: com o limiar 0,5 o custo seria R$ 1.578.000; aprovar todos, R$ 2.016.000; recusar todos, R$ 1.422.000. O limiar escolhido reduz o custo em 32% frente ao 0,5 e em 47% frente a aprovar todos.

Limiar	Precisão	Recall	FN	FP	Custo (teste)
0,10	28,3%	89,3%	27	571	R$ 1.072.500
0,15	31,9%	79,4%	52	427	R$ 1.056.500
0,19	**34,9%**	**71,8%**	**71**	**338**	**R$ 1.075.000**
0,30	46,0%	52,4%	120	155	R$ 1.192.500
0,50	58,9%	25,0%	189	44	R$ 1.578.000

Nota: a tabela do teste serve só para *reportar* o resultado. Nela o limiar 0,15 custaria um pouco menos, mas a escolha foi feita sem o teste (usá-lo para escolher o limiar o tornaria otimista). A diferença (cerca de R$ 18 mil) é pequena e compatível com ruído amostral. O custo no teste (1.200 contratos) é menor que no treino (4.800) por ter 4 vezes menos linhas.

Parte 5 — Integração ao sistema

Foi adicionado o problema `credito` ao dicionário `PROBLEMAS` em `config.py`, com as nove features, limites (mín./máx.), valores padrão e opções das categorias (`possui_imovel`: sim/nao; `finalidade`: pessoal, veiculo, reforma, educacao, negocio). `id_contrato` ficou de fora. **Não foi necessário alterar** `app.py` **nem** `index.html`: como tudo é lido de `config.py`, a API e o formulário passaram a incluir a nova aba automaticamente. Ao rodar `python treinar.py`, o terceiro modelo foi treinado e salvo em `models/credito.joblib`.

As telas usam o limiar 0,5 para a cor da interface; para a decisão de negócio vale o limiar da Parte 4.

Risco alto — 28 anos, renda R$ 1.800, 6 meses de emprego, score 380, 5 dívidas ativas, sem imóvel, finalidade negócio, R$ 30.000 em 12 meses: **98,9%** de probabilidade de inadimplência.



Risco baixo — 45 anos, renda R$ 12.000, 12 anos de emprego, score 850, sem dívidas, com imóvel, educação, R$ 10.000 em 36 meses: **0,4%** de probabilidade de inadimplência.



Parte 6 — Reflexão ética

Modelos de crédito aprendem com decisões do passado; se um grupo já foi preterido por preconceito, o modelo reproduz e escala essa discriminação, ainda com aparência de objetividade. O risco existe mesmo sem variáveis protegidas, porque outras funcionam como *proxies*: CEP ou bairro podem refletir renda, raça e etnia, e a idade pode prejudicar jovens e idosos. Por isso eu **não usaria**, mesmo que melhorassem o desempenho, gênero, raça/cor, religião, orientação sexual, estado civil, nome, dados de saúde nem CEP/bairro; a idade só com justificativa e auditoria (ela está no dataset do exercício, mas eu a revisaria antes de ir a produção). Também mediria taxa de aprovação, recall e falsos positivos por grupo, para detectar tratamento desigual, e preferiria um modelo interpretável como a regressão logística, cujo ganho de desempenho frente aos complexos foi nulo aqui. Pela LGPD (Lei 13.709/2018): o tratamento exige finalidade específica e base legal (a proteção do crédito é uma das hipóteses do art. 7º, X), com coleta apenas dos dados necessários (art. 6º: finalidade, necessidade e não discriminação); dados sensíveis têm regime mais restrito (art. 11); e o art. 20 garante ao titular o direito de pedir a revisão de decisões tomadas unicamente por tratamento automatizado e de obter informações claras sobre os critérios usados. Na prática, o modelo deve apoiar a equipe de crédito (humano no circuito), com explicação do motivo da recusa e canal de contestação, e o desempenho e o viés devem ser monitorados ao longo do tempo.

Desafios extras (bônus)

1. Feature "comprometimento de renda" (parcela estimada = valor × 1,33 / prazo, dividida pela renda). Na validação cruzada o ROC AUC praticamente não mudou: 0,7781 ± 0,0224 sem a feature contra 0,7779 ± 0,0229 com ela. No teste foi de 0,762 para 0,771, diferença pequena e dentro do ruído (o desvio entre folds é ~0,022). Conclusão: nestes dados e com este modelo não há ganho confiável; este experimento sozinho não permite afirmar a causa.

2. GridSearchCV (regressão logística com a nova feature; C em 0,01; 0,1; 1; 10; 100; 5 folds): melhor C = 0.1, AUC de CV 0,7779 e de teste 0,771. Ganho desprezível: o modelo já estava bem regularizado.

3. class_weight="balanced" (com a nova feature, limiar 0,5, no teste):

Configuração	Precisão	Recall	F1	ROC AUC	Custo
Sem class_weight	59,8%	26,6%	0,368	0,771	R$ 1.547.500
balanced	38,0%	68,7%	0,489	0,771	R$ 1.055.000

O balanceamento **troca precisão por recall**: o recall sobe bastante, a precisão cai e o ROC AUC quase não muda, ou seja, o ranqueamento é o mesmo e o efeito equivale a deslocar o limiar para baixo. Como o falso negativo custa mais, o custo melhora em relação ao 0,5 sem balanceamento, mas escolher o limiar pelo custo (Parte 4) é mais direto e controlável.

Arquivos entregues

`config.py` (com `credito`) · `gerar_dados.py` · `treinar.py` · `app.py` · `templates/index.html` · `analise_credito.py` (Partes 1 a 4 e bônus) · `resultados_credito.json` · `figuras/`.
Reprodução: `python gerar_dados.py`, `python treinar.py`, `python analise_credito.py` e `python app.py`.